

Working with Directories

MODULE 1: GETTING STARTED & NAVIGATING THE SYSTEM

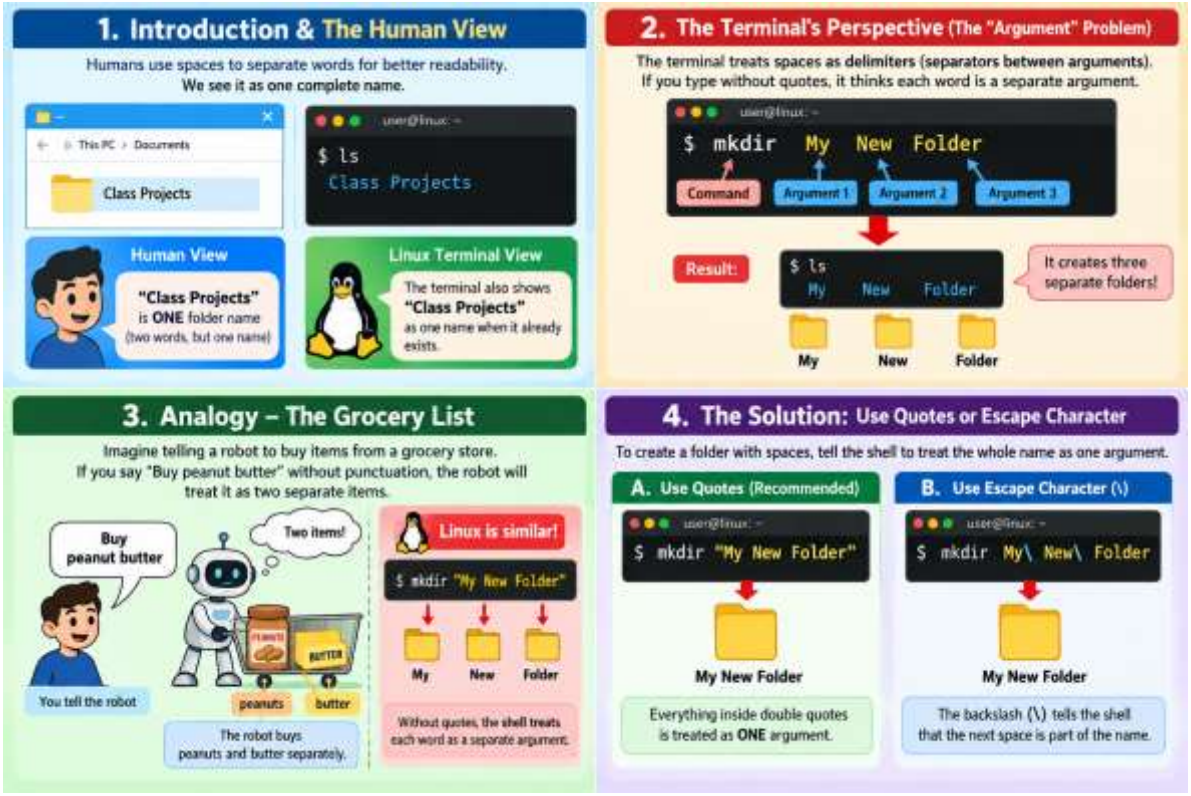
INDEX

- Directory Management in Linux
- Concept of spaces in Linux
- Creating Directories
- Removing Directories
- rm command
- Wildcards in Linux
- Various options with rm command
- Absolute vs Relative Path in Linux

Directory Management in Linux

- Directories in Linux are special types of files that store metadata mappings to other files and directories, organizing the filesystem in a hierarchical structure.
  - Directories act as containers for files and subdirectories.
  - The root directory (/) is the **top-level directory** in Linux.
  - Special directories like . (current directory) and .. (parent directory) are used for navigation.
  - Each directory can contain multiple files and subdirectories.
- In Linux, Managing directories is primarily done through the command-line interface.
- The core operations involve **navigating, listing, creating, moving,** and **deleting directories** using a set of standard utilities.

Concept of Spaces in Linux



Creating Directories

- The *mkdir* command is is used to create new directories (which are also referred to as *folders* in some operating systems). Its syntax is

mkdir [option] directory_name(s)

- To create multiple directories – **mkdir dir1 dir2**
- To create dir3 directly make sure dir1, dir2 available – **mkdir dir1/dir2/dir3**
- To create a directories in the specified path – **mkdir –p dir1/dir2/dir3**

  - **- p** means path of directories
  - **dir1** will be created. In that **dir2** will be created & within that **dir3** will be created
  - The most common use of **–p** in the Linux terminal is with **mkdir**. It creates a directory path automatically.

Removing Directories

- The *rmdir* command provides another way to remove empty directories. Its syntax is:

rmdir [option] directory_name(s)

- **Note: We cannot use rmdir for files. If the directory is not empty, then we will get an error.**
- To remove multiple empty directories,

rmdir dir1 dir2

rm command

- The *rm* command is often the most efficient way to remove files and directories. Its syntax is

rm [option] file_name(s)

- To delete file1, the following command would be used:

rm file1

- To delete both file2 and file3, the following command would be,

rm file2 file3

This can be tedious if it desired to delete a large number of files, such as all files within a directory. A much more efficient (but also more dangerous) way to do this is to use **wildcards**, which can be used to represent anything.

Wildcards in Linux

- In Linux, wildcards (also known as **globbing patterns**) allow you to target multiple files or directories at once based on character patterns rather than typing every filename individually.
- Crucially, the **rm** command itself **does not process wildcards.** Instead, your Linux shell (like Bash or Zsh) automatically expands the wildcard pattern into a complete list of matching filenames before passing them to rm

Common Wildcards Used:

1. **The Asterisk (*)** – Matches Zero or More Characters
2. **The Question Mark (?)** – Matches Exactly One Character
3. **Square Brackets ([])** – Matches a Defined Character Set

Wildcards in Linux – Find Files Using Patterns

Wildcards are special characters that help you match multiple files and directories based on a pattern.

1. * (Asterisk)

Matches zero or more characters.

Analogy:
Like a shopping shelf label **"snack*"**.
It matches all snacks (small, big, any type).



Files in folder:

- file.txt
- file1.txt
- file_backup.txt
- image.jpg

```
$ ls file*
```

Matches:

- file.txt
- file1.txt
- file_backup.txt

2. ? (Question Mark)

Matches exactly one character.

Analogy:
Like a parking slot **"car?"**.
It matches one car with exactly one number.



Files in folder:

- file1.txt
- file2.txt
- file10.txt
- fileA.txt

```
$ ls file?.txt
```

Matches:

- file1.txt
- file2.txt
- fileA.txt

file10.txt is not matched (because it has two characters)

3. [ ] (Character Set)

Matches one character from the given set.

Analogy:
Like a bus stop for routes **1, 2 or 3**.
Only these buses stop here.



Files in folder:

- file1.txt
- file2.txt
- file3.txt
- file4.txt

```
$ ls file[123].txt
```

Matches:

- file1.txt
- file2.txt
- file3.txt

4. [a-z] (Range)

Matches one character in the given range.

Analogy:
Like a shelf for files **A to C**.
Only files with a letter from **a to c** are considered.



Files in folder:

- filea.txt
- fileb.txt
- filec.txt
- filed.txt

```
$ ls file[a-c].txt
```

Matches:

- filea.txt
- fileb.txt
- filec.txt

filed.txt is not matched (because 'd' is not in a-c)

5. [!] (Not)

Matches one character that is NOT in the set.

Analogy:
Like a gate that allows all buses **except No. 1**.



Files in folder:

- file1.txt
- file2.txt
- file3.txt
- file4.txt

```
$ ls file[!1].txt
```

Matches:

- file2.txt
- file3.txt
- file4.txt

file1.txt is not matched (because 1 is excluded)

How Wildcards Work (Behind the Scenes)



1. You type a command with a wildcard.

```
$ ls *.txt
```

2. The shell expands the wildcard to matching filenames.

```
$ ls notes.txt report.txt data.txt
```

3. The command runs with the expanded list.

```
ls notes.txt report.txt data.txt
```

Wildcard Cheat Sheet

Wildcard	Meaning	Example
*	Zero or more characters	`ls *.txt`
?	Exactly one character	`ls file?.txt`
[abc]	One character from a set	`ls file[123].txt`
[a-z]	One character in a range	`ls file[a-c].txt`
[!abc]	One character NOT in a set (Bash)	`ls file[!1].txt`

Various options with rm cmd

Option	Long Option	Description	Example
-f	--force	Ignores nonexistent files and never prompts for confirmation, even if a file is write-protected.	rm -f abc.txt
-i	--interactive=always	Prompts for confirmation before deleting every single file.	rm -i abc.txt
-I	--interactive=once	Prompts only once before deleting more than three files, or when deleting recursively.	rm -I *.log
-r (or) -R	--recursive	Removes directories and all of their contents (subfolders and files) recursively.	rm -r test/
-d	--dir	Removes empty directories only (similar to rmdir).	rm -d dev
-v	--verbose	Explains what is being done, printing the name of each file as it is deleted.	rm -v cache.tmp

Absolute vs Relative Path in Linux

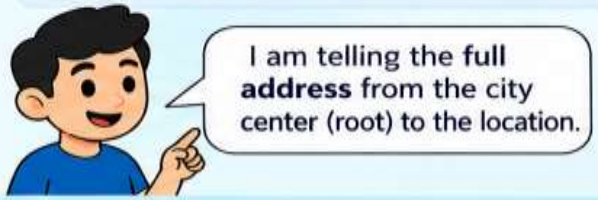
- The primary difference between an absolute and a relative path in Linux is their starting point: an **absolute path** always starts from **the root directory (/)**, while a **relative path** starts from your **current working directory**.

Feature	Absolute Path	Relative Path
Starting Point	The system root directory (/)	Your current working directory (pwd)
Syntax Start	Always begin with a forward slash /	Never begins with a forward slash /
Location Dependency	Independent of where you currently are	Changes meaning depending where you are
Best Used for	System configurations, cron jobs, and automation scripts	Interactive terminal navigation and portable code projects

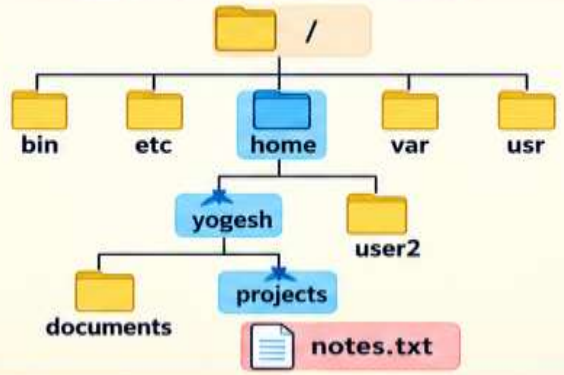
Absolute Path vs Relative Path in Linux

1. Absolute Path

An absolute path is the **complete path** from the **root directory (/)** to the file or folder.
It **always starts with** / and is the same no matter where you are in the system.

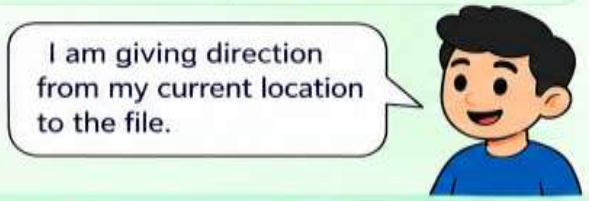


Linux File System (Example)



2. Relative Path

A relative path is the path from your **current working directory** to the file or folder.
It **does not start with /.**
It **depends on where you are currently.**



Absolute Path Example

```
yogesh@linux:~$ cat /home/yogesh/projects/notes.txt
```

Starts with / (root)
Complete path

Meaning:
Go to / → home → yogesh → projects → notes.txt and open the file.

Path Comparison

Feature	Absolute Path	Relative Path
Starts with	/ (root)	Does not start with /
Path type	Complete path from root	From current directory
Example	/home/yogesh/projects/notes.txt	**notes.txt** or projects/notes.txt
Works from anywhere	Yes	No (depends on current location)
Like in real life	Full address with city, area, street	Direction from your current place

Relative Path Examples

1 If you are inside /home/yogesh

```
yogesh@linux:~$ cd projects
yogesh@linux:~/projects$ cat notes.txt
```

Current directory is /home/yogesh/projects So we can directly use the file name.

2 If you are inside /home/yogesh

```
yogesh@linux:~$ cat projects/notes.txt
```

Path from current directory (/home/yogesh) to notes.txt.

3 If you are inside /home

```
yogesh@linux:/home$ cat yogesh/projects/notes.txt
```

Path from current directory (/home) to notes.txt.

Real-Life Analogy

Absolute Path
(Full Address)



123, MG Road,
Chennai, Tamil Nadu, India

Complete address from the city center.
You can use this no matter where you are.

Relative Path
(Direction from current location)



From this building, go 2 streets, then left.



Direction from where you are now.
It depends on your current location.